

Paper

“latex

1 Summary

Bài báo *ProRL: Effective Reinforcement Learning for Proactive Recommendation via Rectified Policy Gradient Estimation* nghiên cứu bài toán *Proactive Recommender Systems* (PRS). Khác với recommender system truyền thống, vốn chủ yếu phản ánh sở thích hiện tại của user, PRS hướng đến việc chủ động dẫn dắt user khám phá một target item mà nền tảng muốn giới thiệu. Thay vì gợi ý trực tiếp target item, hệ thống tạo ra một chuỗi item trung gian để dần dịch chuyển sở thích của user từ vùng quen thuộc sang vùng mục tiêu.

Ví dụ, nếu user đang thích phim khoa học viễn tưởng nhưng platform muốn dẫn user đến phim hài, hệ thống có thể không gợi ý ngay phim hài thuần túy. Thay vào đó, nó tạo một path như *Sci-Fi + Animation* $\rightarrow$ *Animation + Comedy* $\rightarrow$ *Comedy*. Mỗi item trung gian vẫn đủ gần với sở thích hiện tại để user có khả năng chấp nhận, nhưng toàn bộ path sẽ dần kéo user về phía target preference.

Bài toán này có hai mục tiêu chính. Thứ nhất là *Path Feasibility*, tức mỗi item trung gian trong path phải có xác suất được user chấp nhận đủ cao. Nếu item trung gian quá xa sở thích hiện tại, user có thể bỏ qua và quá trình guidance thất bại. Thứ hai là *Guidance Effectiveness*, tức sau khi đi qua path, xác suất user quan tâm đến target item phải tăng lên đáng kể. Hai mục tiêu này cần được tối ưu đồng thời vì một path dễ được click chưa chắc đã dẫn user đến target item, còn một path hướng mạnh đến target nhưng quá xa sở thích hiện tại thì lại thiếu khả thi.

Về mặt kỹ thuật, bài báo sử dụng một *user simulator* để đánh giá path. User simulator là một recommender model đã được huấn luyện trên lịch sử tương tác, ví dụ SASRec, dùng để ước lượng xác suất user chấp nhận item i khi đang có interaction sequence S . Ký hiệu xác suất này là:

$$P(i \mid S)$$

Nhờ simulator, mô hình có thể đánh giá chất lượng path mà không cần thử trực tiếp với user thật trong môi trường online.

Bài báo định nghĩa ba metric chính để đo chất lượng path. Metric thứ nhất là *Increment of Interest* (IoI), đo mức tăng log-probability của target item sau khi thêm path vào lịch sử user:

$$IoI = \log P(i_T | S_u \oplus L_u) - \log P(i_T | S_u)$$

Trong đó S_u là lịch sử tương tác ban đầu của user, L_u là recommendation path, i_T là target item, và $\oplus$ là phép nối chuỗi.

Metric thứ hai là *Increment of Rank* (IoR), đo mức cải thiện thứ hạng của target item sau khi user đi qua path:

$$IoR = \text{Rank}(i_T | S_u) - \text{Rank}(i_T | S_u \oplus L_u)$$

Nếu thứ hạng của target item giảm từ 1000 xuống 200, thì IoR dương và lớn, nghĩa là path đã giúp target item trở nên phù hợp hơn với user.

Metric thứ ba là *Click-Through Rate* (CTR), đo feasibility của path bằng trung bình xác suất user chấp nhận từng item trung gian:

$$CTR = \frac{1}{|L_u|} \sum_{k=1}^{|L_u|} P(i_k | S_u \oplus L_u^{<k})$$

Trong đó i_k là item thứ k trong path, còn $L_u^{<k}$ là các item trước đó trong path. CTR đảm bảo rằng path không chỉ hiệu quả ở cuối, mà từng bước đi cũng phải khả thi.

Từ ba metric trên, bài báo định nghĩa path-level reward:

$$R_{path} = \alpha \cdot IoI + \beta \cdot IoR + \gamma \cdot CTR$$

Trong đó α, β, γ là trọng số của từng mục tiêu. Như vậy, proactive recommendation được đưa về bài toán tối đa hóa reward của một chuỗi hành động, rất phù hợp với reinforcement learning.

Policy $\pi_\theta(\cdot | S_u, i_T)$ nhận đầu vào là lịch sử user và target item, sau đó sinh ra recommendation path $L_u = (i_1, \dots, i_L)$. Policy ban đầu được khởi tạo bằng supervised pretraining trên các path lịch sử, sau đó được fine-tune bằng reinforcement learning. Objective của RL là:

$$J(\theta) = \mathbb{E}_{L_u \sim \pi_\theta} [R_{path} - \lambda D_{KL}(\pi_\theta \| \pi_0)]$$

Trong đó π_0 là pretrained policy, còn KL term giúp policy sau RL không lệch quá xa khỏi distribution ban đầu.

Nếu dùng policy gradient chuẩn, gradient estimator có dạng:

$$\hat{g}_{std} = \frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m \left[\sum_{t=1}^{L^{(i,j)}} \nabla_{\theta} \log \pi_{\theta}^{(i,j,t)} \cdot R^{(i,j)} \right]$$

Trong đó mỗi step trong path đều được nhân với cùng một path-level reward. Về lý thuyết, cách này có thể tối ưu reward. Tuy nhiên, bài báo chỉ ra rằng khi áp dụng trực tiếp vào PRS, policy gradient chuẩn bị lỗi nghiêm trọng.

Lỗi thứ nhất là *Length Shortcut*. Path-level reward có thể phân rã thành tổng các step-level reward:

$$R(i_1, \dots, i_L) = \sum_{t=1}^L r_t$$

với:

$$r_t = R(i_1, \dots, i_t) - R(i_1, \dots, i_{t-1})$$

Bài báo quan sát rằng các step-level reward thường có kỳ vọng dương. Khi đó, path càng dài thì expected reward càng cao. Điều này tạo ra bias khiến policy học cách kéo dài path thay vì học cách chọn item trung gian thật sự chất lượng. Kết quả là policy nhanh chóng sinh ra các path rất dài, gần chạm L_{max} , và các path này trở nên giống nhau, làm giảm diversity.

Lỗi thứ hai là *High Gradient Variance*. Policy gradient chuẩn nhân gradient ở mọi step với toàn bộ path-level reward. Nhưng hành động ở step t chỉ ảnh hưởng đến reward từ step t trở đi, không ảnh hưởng đến các reward trước đó. Vì vậy, dùng toàn bộ reward cho mọi step sẽ đưa nhiều nhiễu không liên quan vào gradient, làm variance cao và khiến quá trình học kém ổn định.

Để giải quyết hai lỗi này, bài báo đề xuất framework *ProRL*. ProRL không thay đổi hoàn toàn bài toán PRS, mà rectifies policy gradient estimation bằng hai cơ chế chính: *Stepwise Reward Centering* (SRC) và *Position-Specific Advantage Estimation* (PSAE).

Cơ chế thứ nhất, *Stepwise Reward Centering*, nhằm loại bỏ length shortcut. Ý tưởng là trừ đi expected step reward để việc kéo dài path không còn tự động tạo thêm expected gain. Với mỗi step reward r_t , ProRL định nghĩa centered reward:

$$\tilde{r}_t = r_t - \bar{r}$$

trong đó:

$$\bar{r} = \mathbb{E}_{\pi}[r_*]$$

là expected step reward toàn cục, được ước lượng từ rollout. Khi đó:

$$\mathbb{E}_{\pi}[\tilde{r}_t] = 0$$

nên việc thêm step mới vào path không còn tạo lợi thế kỳ vọng chỉ vì path dài hơn. Gradient lúc này phải đến từ chất lượng thực sự của item được chọn, thay vì từ độ dài path.

Với multi-objective reward gồm nhiều thành phần như CTR, IoI và IoR, các reward component có scale khác nhau. Vì vậy, bài báo mở rộng SRC thành dạng normalization:

$$\tilde{r}_t = \sum_{i=1}^K w_i \cdot \frac{r_t^{(i)} - \mu^{(i)}}{\sigma^{(i)}}$$

Trong đó $\mu^{(i)}$ và $\sigma^{(i)}$ là mean và standard deviation của step reward component thứ i . Cách này vừa center reward để loại bỏ length shortcut, vừa đưa các component về scale tương đối đồng đều để multi-objective optimization ổn định hơn.

Cơ chế thứ hai, *Position-Specific Advantage Estimation*, nhằm giảm gradient variance. Thay vì dùng toàn bộ path reward R cho mọi step, ProRL dùng *reward-to-go*:

$$G_t = \sum_{\ell=t}^L r_\ell$$

Reward-to-go chỉ tính reward từ step t trở đi, loại bỏ các reward trong quá khứ vốn không bị ảnh hưởng bởi hành động tại step t .

Sau đó, ProRL tiếp tục trừ baseline theo từng vị trí. Với nhiều rollout từ cùng một input, baseline tại position t được tính bằng trung bình reward-to-go của các path đạt đến vị trí đó:

$$\bar{G}_{i,t} = \frac{\sum_{j:L^{(i,j)} \geq t} G_t^{(i,j)}}{\sum_{j=1}^m \mathbb{I}[L^{(i,j)} \geq t]}$$

Position-specific advantage được định nghĩa là:

$$\hat{A}_t^{(i,j)} = G_t^{(i,j)} - \bar{G}_{i,t}$$

Khác với GRPO dùng một baseline chung ở cấp path, PSAE dùng baseline riêng cho từng vị trí trong path. Điều này phù hợp với PRS vì expected future reward ở step đầu, step giữa và step cuối thường rất khác nhau.

Gradient estimator sau khi rectified có dạng:

$$\hat{g}_{rect} = \frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m \left[\sum_{t=1}^{L^{(i,j)}} \nabla_{\theta} \log \pi_{\theta}^{(i,j,t)} \cdot \hat{A}_t^{(i,j)} \right]$$

Estimator này vẫn giữ tinh thần policy gradient, nhưng tín hiệu gradient đã được sửa để tránh length shortcut và giảm variance.

Thực nghiệm được tiến hành trên ba dataset: MovieLens-1M, Steam và Amazon-

Book. Các baseline gồm sequential recommendation models như GRU4Rec, BERT4Rec, LightSANS và FEARec; supervised proactive method như IRN; heuristic proactive methods như IPG và ITMPRec; và LLM-based proactive methods như LLM-IPP và T-PRA. Các metric đánh giá gồm CTR cho path feasibility, IoI và IoR cho guidance effectiveness, cùng với Coherence để đo độ nhất quán ngữ nghĩa giữa các item liên tiếp trong path.

Kết quả cho thấy ProRL đạt hiệu năng tốt nhất trên hầu hết các metric và dataset. Đặc biệt, ProRL không chỉ tăng IoI và IoR, tức khả năng dẫn user đến target item, mà còn giữ CTR cao, nghĩa là các item trung gian vẫn có khả năng được user chấp nhận. Coherence cũng tăng dù không được đưa trực tiếp vào reward, cho thấy ProRL không chỉ exploit reward mà còn học được path tự nhiên và hợp lý hơn.

Bài báo cũng thực hiện cross-evaluator analysis để kiểm tra liệu ProRL có overfit vào user simulator hay không. Policy được train với một evaluator, ví dụ SASRec, sau đó được đánh giá bằng các evaluator khác như GRU4Rec, LightSANS và BERT4Rec. Kết quả vẫn cải thiện ổn định, cho thấy ProRL học được guidance strategy có khả năng generalize, chứ không chỉ reward hacking trên một simulator cụ thể.

Ablation study xác nhận vai trò của hai module chính. Khi bỏ Stepwise Reward Centering, mô hình có thể đạt CTR rất cao nhưng IoI và IoR giảm mạnh, nghĩa là nó ưu tiên click ngắn hạn thay vì guidance dài hạn. Khi bỏ Position-Specific Advantage Estimation, hiệu quả guidance cũng giảm do gradient variance cao hơn. Ngoài ra, ablation trên reward design cho thấy CTR, IoI và IoR bổ sung cho nhau; bỏ bất kỳ thành phần nào cũng làm giảm chất lượng tổng thể.

2 Conclusion

Hướng đi đã làm của bài báo là đưa proactive recommendation về bài toán reinforcement learning, trong đó policy sinh ra một path các item trung gian để dần dịch chuyển sở thích user đến target item. Thay vì chỉ học bất chước path lịch sử bằng supervised learning, ProRL dùng reward gồm CTR, IoI và IoR để trực tiếp tối ưu cả path feasibility và guidance effectiveness. Đây là một hướng hợp lý vì proactive recommendation bản chất là bài toán sequential decision-making.

Đóng góp kỹ thuật quan trọng nhất của bài báo là phát hiện rằng policy gradient chuẩn không phù hợp trực tiếp với PRS do hai lỗi: *length shortcut* và *high gradient variance*. Length shortcut khiến mô hình học cách kéo dài path để tăng reward kỳ vọng, thay vì học path chất lượng. High variance khiến gradient ở từng step bị nhiễu vì dùng toàn bộ path reward cho mọi action. Hai vấn đề này làm RL dễ suy thoái thành các path dài, giống nhau và kém hiệu quả.

ProRL giải quyết hai vấn đề trên bằng hai cơ chế rectification. *Stepwise Reward Centering* trừ expected step reward để path extension có expected gain bằng 0, từ đó loại bỏ động cơ kéo dài path vô nghĩa. *Position-Specific Advantage Estimation* dùng

reward-to-go và baseline theo từng vị trí để giảm variance, giúp mỗi action được đánh giá theo phần reward tương lai mà nó thực sự ảnh hưởng. Nhờ đó, gradient tập trung hơn vào path quality.

Kết quả thực nghiệm cho thấy ProRL vượt trội hơn các phương pháp supervised, heuristic, sequential recommendation và LLM-based proactive recommendation. Điều này cho thấy RL có thể là hướng mạnh cho PRS nếu gradient estimation được thiết kế đúng với cấu trúc reward của bài toán. Ngoài ra, cross-evaluator analysis cho thấy policy học được không chỉ tối ưu cho một simulator cụ thể mà có khả năng chuyển giao sang các evaluator khác.

Hướng kế thừa từ bài báo có thể gồm nhiều nhánh. Thứ nhất, có thể kết hợp ProRL với các mô hình user simulator mạnh hơn, ví dụ Transformer-based recommender, generative recommender hoặc multi-interest model, để reward phản ánh user behavior chính xác hơn. Thứ hai, có thể mở rộng reward bằng các yếu tố như novelty, diversity, fairness, long-term retention hoặc user satisfaction để tránh việc hệ thống chỉ tối ưu target item theo lợi ích ngắn hạn. Thứ ba, có thể đưa thêm temporal/contextual information như thời điểm, session, device, location hoặc mood để proactive path không chỉ đúng về item, mà còn đúng về ngữ cảnh sử dụng.

Ngoài ra, bài báo này cũng có thể kế thừa cho hướng recommender có tính thời điểm hoặc chu kỳ. Thay vì chỉ dẫn user đến target item bất kỳ lúc nào, hệ thống có thể học policy sinh guidance path phụ thuộc vào thời điểm: ví dụ dẫn user đến sản phẩm mùa vụ, nội dung theo dịp lễ, hoặc item đang trong campaign promotion. Khi kết hợp ProRL với các temporal pattern như periodic pattern, evolving pattern hoặc holistic temporal pattern, hệ thống có thể tối ưu đồng thời ba yếu tố: gợi ý đúng item, đúng lộ trình, và đúng thời điểm. ““